Viva Preparation Guide

Two From-Scratch Monolingual Transformers
Hindi (Model H) and Nepali (Model L)

Explaining, defending and deriving every part of the project

Swaraj Kumar

Personal study document — not a graded deliverable

October 8, 2026

What this document is, and how to use it

This is my own preparation guide for the final viva. It is **not** a course deliverable — the graded report is `report/final/final_report.pdf`.
Every number in here is the same number as in that report, and both are checked against the actual result files by `report/final/check_report_facts.py`, which fails if a value drifts away from the artifact it came from. It also verifies that every function and class name cited here really exists in the repository.

How to use it. Read Chapter 1 and the cheat sheet first, then work through Chapters 3–5. Chapters 9, 10 and 11 — design decisions, limitations and trap questions — are where examiners actually spend their time. Do not memorise: the *Think of it like this* boxes exist so that you can rebuild an answer from understanding when the exact wording has gone.

Contents

1 The Two-Minute Introduction **4**
- 1.1 30-second version . . . 4
- 1.2 1-minute version . . . 4
- 1.3 2-minute version . . . 4

2 The Complete Project Pipeline **6**
- 2.1 The evidence chain . . . 6
- 2.2 Every stage: what, why, how, output . . . 6

3 Phase 1 — Corpora and Tokenizers **8**
- 3.1 Why these two languages? . . . 8
- 3.2 Manual versus downloaded data . . . 8
- 3.3 Cleaning and normalisation . . . 9
  - 3.3.1 Why NFC and not NFKC? . . . 10
  - 3.3.2 Zero-width characters, ZWNJ and ZWJ . . . 10
  - 3.3.3 Language identification . . . 11
- 3.4 Deduplication — exact versus near . . . 11
- 3.5 Shingles, Jaccard, MinHash and LSH . . . 12
- 3.6 Splits, and why document-level . . . 13
- 3.7 Tokenizers . . . 14
  - 3.7.1 BPE and Unigram . . . 14
  - 3.7.2 The measured sweep . . . 15
  - 3.7.3 Why separate tokenizers and no shared vocabulary? . . . 16
- 3.8 What I would improve in Phase 1 . . . 16

4 Phase 2 — Architecture, Pretraining, Evaluation **17**
- 4.1 The architecture . . . 17
  - 4.1.1 Why decoder-only? . . . 17
- 4.2 Attention . . . 18
  - 4.2.1 Why divide by $\sqrt{d_k}$? . . . 18
- 4.3 Causal masking . . . 19
- 4.4 The rest of the block . . . 20
  - 4.4.1 Pre-norm versus post-norm . . . 20
  - 4.4.2 Feed-forward network and GELU . . . 20
  - 4.4.3 Residual connections . . . 21
- 4.5 Parameter count — derive this live . . . 21
  - 4.5.1 Weight tying . . . 22
  - 4.5.2 Positional embeddings . . . 22
- 4.6 Training . . . 23
  - 4.6.1 Checkpointing and resume . . . 23
- 4.7 Intrinsic results . . . 24
- 4.8 Generation . . . 24
- 4.9 Attention analysis . . . 25

5 Phase 3 — Reasoning Finetuning **27**
5.1 Why synthetic data? . . . 27
5.2 The four task families . . . 27
5.2.1 Why `transitive3` is the real test . . . 28
5.3 Dataset construction and leakage control . . . 28
5.3.1 The gender-agreement leak . . . 29
5.4 Finetuning protocol . . . 29
5.5 Exact match versus forced choice . . . 30
5.6 Error analysis . . . 31
5.7 Overfitting and checkpoint selection . . . 32
5.8 Attention before and after finetuning . . . 32

6 Cross-Phase Questions **34**

7 Code-Level Questions **36**
7.1 Where everything lives . . . 36
7.2 Likely code questions . . . 37

8 Results and Metrics **39**
8.1 Perplexity versus bits-per-byte . . . 39
8.2 Numbers that prove things . . . 40

9 "Why Did You Make This Choice?" **41**

10 What Went Wrong — Limitations and Lessons **43**

11 Trap and Follow-Up Questions **44**

12 Rapid-Fire **46**

13 The Final One-Minute Conclusion **48**

A Final Cheat Sheet **49**
A.1 Must-memorise numbers . . . 49
A.2 Formula sheet . . . 50
A.3 Ten things I must NOT claim . . . 50

Chapter 1

The Two-Minute Introduction

1.1 30-second version

Say it like this

I built two separate decoder-only Transformers from scratch in PyTorch, one for Hindi and one for Nepali. Nothing is shared between them except the source code — separate corpora, separate tokenizers, separate weights. Both are about 23 million parameters. The interesting result is that Hindi looks 41% better on perplexity, but that advantage mostly disappears when you measure in bits-per-byte, because the two tokenizers chop text into different-sized pieces.

1.2 1-minute version

Say it like this

The task was to build two monolingual language models of about 25M parameters each, with no pretrained weights and no pretrained tokenizers. I picked Hindi as the higher-resource language and Nepali as the lower-resource one, and I picked that pair deliberately because both use Devanagari — if the script is the same and the architecture is the same, then any difference between the models has to come from the data or the tokenizer, not from the writing system or model size.

Phase 1 was collecting and cleaning the corpora and training the tokenizers: 506 million Hindi tokens and 635 million Nepali, both with over 20% manually collected. Phase 2 was implementing the Transformer myself — attention, causal masking, positional embeddings — and pretraining both models for about 475 million tokens on a Kaggle T4. Phase 3 was generating synthetic comparative-reasoning data in each language and finetuning on it.

The headline results are that Hindi gets test perplexity 27.41 and Nepali 46.68, but on bits-per-byte it is 0.5382 against 0.4846 — the ranking flips. And in Phase 3, exact match went from 0% to 35%, but when I forced the models to choose between the names in the question they were at chance, so they learned the answer format rather than the comparison.

1.3 2-minute version

Say the one-minute version, then add:

Add this for the two-minute version

The part I would most want to talk about is that this project gave me the same trap twice, in opposite directions. In Phase 2, if I had only reported perplexity I would have concluded the

low-resource model was far worse — bits-per-byte showed most of that gap was the tokenizer. In Phase 3, if I had only reported exact match I would have concluded the models learned to reason — forced choice showed they had not. Both times a second metric measuring the same thing in different units is what caught it.

I also traced two Phase 1 decisions forward. Nepali's tokenizer has no byte fallback, which showed up as a column of zeros that I misread as a good score; two phases later it meant the Nepali vocabulary had no Devanagari digit at all, so 4.8% of my generated reasoning data turned into unknown tokens. And Hindi only deduplicates on exact SHA-1 hashes while Nepali does MinHash near-duplicate removal plus train/test decontamination — which is a real weakness in the Hindi pipeline and a limit on how strongly I can interpret the comparison.

Key point

Lead with the controlled-comparison design — same script, same architecture. That one sentence is what makes the whole project sound deliberate rather than assembled.

Chapter 2

The Complete Project Pipeline

2.1 The evidence chain

Every stage below was run **twice and independently**, once per language. Only the source code is shared.

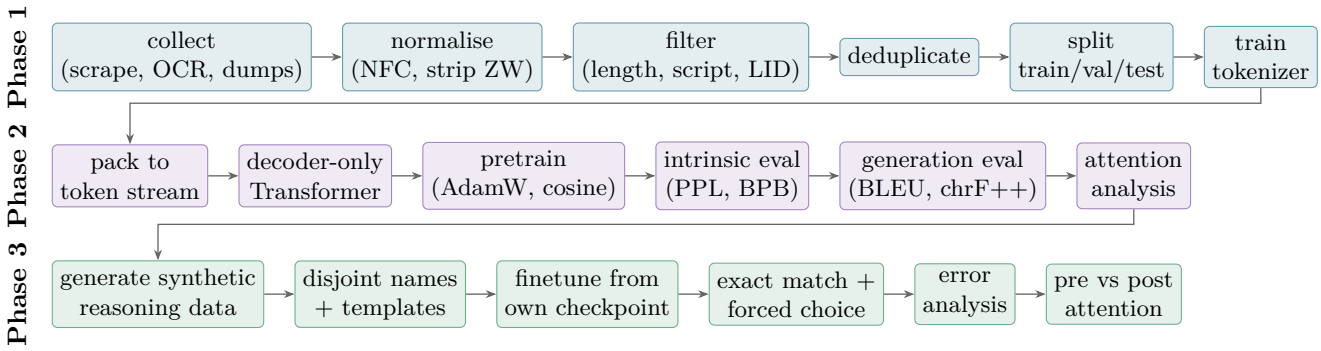


Figure 2.1: The two vertical arrows are the ones that carry Phase 1 decisions into later measurements. Chapter 6 shows that both of them did.

2.2 Every stage: what, why, how, output

Stage	What	Why	How	Output
Collection	Gather raw text per language	Need ≈500M tokens, ≥20% manual	Scraping, Wikipedia dumps, OCR, public corpora	773,606 + 2,655,504 raw docs
Normalisation	Make text canonical	One string, two encodings defeats dedup	`normalize_text()` — four operations	Byte-canonical text
Filtering	Drop junk	Short, wrong-language or corrupt docs teach nothing	`devanagari_ratio()`, `malformed_ratio()`, `langdetect_verdict()`	H: 704,103 docs kept
Dedup	Remove repeats	Duplicates inflate data and leak into test	H: `dedup_key()`; L: `MinHasher` + LSH	H: 2,983; L: 2,350 + 49,734
Splitting	Train/val/test	Need held-out data	Document-level, seeded	H 96/2/2, L 85/7.5/7.5
Tokenizer	Text → IDs	Model works on IDs	SentencePiece on train split only	Unigram 8k, BPE 10k

Stage	What	Why	How	Output
Packing	Flatten to stream	Training reads contiguous windows	`CorpusPacker.pack_splits()`	435,956,678 / 521,426,359
Transformer	The model	The thing being built	`GPT` in `transformer.py`	23.27M / 24.30M params
Pretraining	Learn the language	Next-token prediction	`Trainer.train()`	≈475M tokens each
LM eval	Measure quality	PPL/BPB on held-out text	`perplexity_and_bpb()`	PPL 27.41 / 46.68
Generation	Produce text	Check fluency, not just loss	`GPT.generate()`	chrF++ 16.29 / 18.05
Attention	Inspect heads	I wrote attention, so I should inspect it	`summarise_model_attention()`	entropy 3.2047 / 3.0898
Reasoning data	Synthetic task	Known ground truth, controlled leakage	`ReasoningGenerator`	20k/2k/2k per language
Finetuning	Adapt to task	Learn task without destroying the LM	`FineTuner.train()`	best val at step 60
Reasoning eval	Measure it	One metric is not enough	`ReasoningEvaluator`	EM 35.15/29.45%
Error analysis	Explain failures	Accuracy hides the mechanism	`position_analysis()`	middle-entity bias 72.6%

Chapter 3

Phase 1 — Corpora and Tokenizers

3.1 Why these two languages?

Short answer — say this

Hindi as the higher-resource language because it has abundant public text, and Nepali from the allowed lower-resource list. I chose that specific pair because both are written in Devanagari — if the script is held constant and I also hold the architecture constant, then any gap between the models has to come from data or tokenizer, not from the writing system or capacity.

If the examiner asks for more

Bhojpuri was my original Model L. I built the complete pipeline for it and it passed validation, but public Bhojpuri text ran out at **84,217,032 tokens** — only 16.8% of the 500M target. The brief allows a documented shortfall, but once the Nepali pipeline cleared its targets I switched rather than submit a corpus one-sixth of the required size.

Common follow-up

"Isn't it a problem that they share a script?" — No, it is the point. If they used different scripts I could never separate "this script is harder" from "this data is worse."

3.2 Manual versus downloaded data

The brief required at least 20% of final training tokens from manual collection. Hindi came out at 30.66% and Nepali at 26.41%.

	Model H (Hindi)	Model L (Nepali)
Manual tokens	155,104,651 (30.66%)	167,787,251 (26.41%)
Downloaded tokens	350,747,377	467,417,082
Total	**505,852,028**	**635,204,333**

Hindi manual sources: self-extracted Wikipedia XML dump (66.7M), Gadya Kosh prose (59.9M), patrika.com archive (19.5M), Kavita Kosh poetry (8.1M), 15 surviving archive.org items of 67 collected (0.85M). **Downloaded:** FineWeb-2 `hin_Deva` and four AI4Bharat Sangraha partitions.

Nepali manual sources: an `onlinekhabar` scrape (91.6% of the manual lane), a self-parsed Wikipedia dump, BBC Nepali, Wikibooks, Wiktionary, Ekantipur, Ratopati, Nagarik, and OCR of archive.org books. **Downloaded:** AI4Bharat Sangraha and IndicCorpV2.

If the examiner asks for more

I quarantined the HuggingFace mirror of Hindi Wikipedia rather than using it, because it is the same text as the dump I had already extracted — counting both would have double-counted. I also checked `robots.txt` per site before scraping and recorded the verdict in `scripts/manual_sites.json`; bhaskar.com and amarujala.com disallow automated access, so I did not scrape them.

Likely challenge — raise it before they do

"Is downloading a Wikipedia XML dump really manual collection?"

Say: "I counted it as manual because I downloaded the raw dump and ran the extraction and cleaning myself, rather than pulling a prepared corpus. I can see the argument that it is closer to a download. If you exclude it, my Hindi manual share drops from 30.66% to **17.47%**, which would be below the floor — so it is the single assumption my Phase 1 compliance rests on."

Saying this before they say it is far stronger than being caught by it.

3.3 Cleaning and normalisation

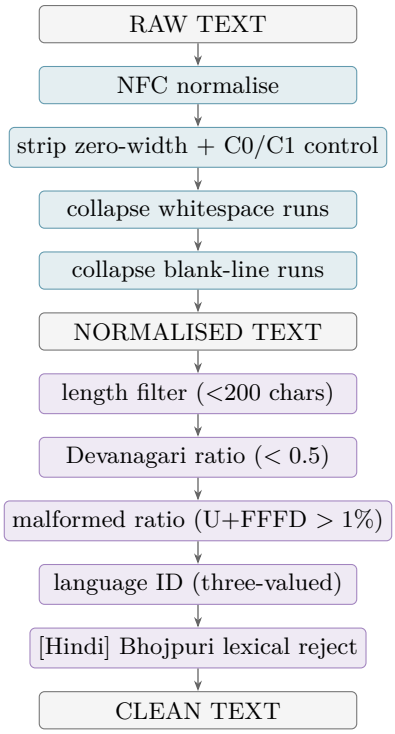


Figure 3.1: The cleaning pipeline. The first five boxes are `normalize_text()` in `scripts/clean_lib.py`; the rest are independent filters.

A document going through normalisation

Raw: नमस्ते दुनिया followed by three newlines, plus a stray U+200E
↓ NFC compose
↓ strip U+200E (left-to-right mark)
↓ collapse the run of spaces to one
↓ collapse three newlines to a paragraph break
Clean: नमस्ते दुनिया

What changed and why: the invisible direction mark is gone — it carries no meaning here and would make two otherwise identical documents hash differently. The extra spacing is normalised so formatting variants collide during deduplication.

3.3.1 Why NFC and not NFKC?

Short answer — say this

NFC is canonical composition — it picks one byte representation for strings that are semantically identical. NFKC goes further and applies *compatibility* mappings, which actually change characters. I only want to canonicalise, not to rewrite the text.

If the examiner asks for more

NFKC would fold ligatures and alter some presentation forms. In Devanagari that risks destroying information I want the model to learn. NFC is the safe choice: same visible text, one canonical encoding.

Why it matters specifically here: the same Devanagari string can be stored two different ways. If I do not normalise before hashing, those two forms produce different hashes, both survive deduplication, and I end up with duplicates I believe I removed. NFC is therefore not cosmetic — it is a **precondition for deduplication working at all**.

Key point

NFC = same text, one encoding. NFKC = possibly different text. I needed the first.

3.3.2 Zero-width characters, ZWNJ and ZWJ

Short answer — say this

I strip U+200E, U+200F, U+FEFF and the C0/C1 control range, but I deliberately **keep** ZWNJ (U+200C) and ZWJ (U+200D).

If the examiner asks for more

In Devanagari, ZWNJ and ZWJ change what is actually rendered — they control whether consonants form a conjunct ligature or stay separate. Removing them would silently change

the word. The others (direction marks, byte-order mark) carry no linguistic content in this corpus and only create spurious hash differences.

Think of it like this

A byte-order mark is *packaging*. A ZWNJ is *spelling*.

3.3.3 Language identification

Short answer — say this

`langdetect_verdict()` returns **three** values — True, False, or `None` for "cannot tell" — with a minimum probability of 0.85. A document is only dropped on a confident False, never on uncertainty.

If the examiner asks for more

Treating "undetermined" as "wrong language" would throw away short or unusual-but-valid documents. Nepali uses a fastText model (`lid.176.bin`) with its own protections in `nepali/scripts/corpuslib/langid.py`: `LanguageIdentifier` calls the raw binding directly, runs `self_test()` on known Hindi/Nepali/English strings at construction, and raises if the error rate crosses a threshold.

Worth volunteering — a silent failure I found and fixed

An earlier Nepali run was destroyed in exactly this area. The `fasttext` wheel calls `np.array(probs, copy=False)`, which raises on NumPy $\geq$2. Every prediction threw, the caller swallowed it as "not our language", and millions of documents were deleted *while the stage logged SUCCESS*. I noticed because the surviving fraction was 5.85%, and it was exactly the one source configured to skip language ID.

That is why the current code bypasses the broken wrapper, self-tests at construction, and counts errors separately from rejections. **Lesson:** a bare `except Exception` around a classifier turns a library bug into silent data loss.

3.4 Deduplication — exact versus near

Know this cold — the two pipelines are different

Do not say "I used MinHash" as a project-wide statement. It is false for Hindi, and an examiner who opens `scripts/clean_lib.py` will find out.

	Model H (Hindi)	**Model L (Nepali)**
Method	SHA-1 over normalised text (`dedup_key()`)	exact `doc_id`, then MinHash + LSH
Catches	exact copies, whitespace and punctuation variants	the above **plus** near duplicates
Removed	2,983 (0.4%)	2,350 exact + 49,734 near
Decontamination	**none**	17,111 train docs removed, rescan clean

Short answer — say this

Hindi deduplicates with a SHA-1 hash of the normalised, punctuation-stripped, case-folded text. That catches exact copies and formatting variants but by construction cannot catch two documents that say the same thing in different words. Nepali uses MinHash with LSH and additionally decontaminates train against validation and test.

The honest admission — prepare it word for word

"Hindi having no near-duplicate removal and no decontamination is a real weakness. It means Hindi's test set may overlap its training data in ways Nepali's cannot, and since the direction of that bias flatters Hindi, part of the perplexity gap I attribute to tokenizer granularity could be unmeasured leakage. I did not quantify it. It is the first thing I would fix."

3.5 Shingles, Jaccard, MinHash and LSH

Work through this until you can reproduce it on paper.

Jaccard similarity by hand

Doc A: "the cat is sitting on the mat"
Doc B: "the cat is sitting on a mat"

Using **word** shingles of size 3 (my code uses 5-*character* shingles; words are easier to show):

A: {the cat is, cat is sitting, is sitting on, sitting on the, on the mat}
B: {the cat is, cat is sitting, is sitting on, sitting on a, on a mat}

$$|A \cap B| = 3, \qquad |A \cup B| = 7, \qquad J(A,B) = \frac{3}{7} = 0.43$$

A and B have *completely different* SHA-1 hashes but are 43% similar — a near duplicate that exact matching silently keeps.

1. **Why exact hashing is not enough.** Change one word and the hash changes completely. See the example above.
2. **What Jaccard means.** $|A \cap B|/|A \cup B|$ — the fraction of distinct pieces the two documents share. 0 = nothing in common, 1 = identical.
3. **Why MinHash.** Comparing Jaccard directly means storing every shingle of every document and intersecting sets — far too slow for 2.2 million documents. MinHash replaces each document

with a short fixed-length signature (mine is **100** numbers) with a useful property: **the probability that two signatures agree in any one position equals their Jaccard similarity**. So I estimate similarity by comparing 100 integers instead of thousands of shingles.

4. **What LSH does.** Even with signatures, all-pairs comparison is $O(n^2)$ — about 2.4×10^{12} pairs at my scale. LSH splits each 100-number signature into **20 bands of 5 rows**, hashes each band, and only compares documents that collide in *at least one* band. Similar documents collide with high probability; dissimilar ones almost never do.

Think of it like this

MinHash is a *fingerprint*. LSH is *filing the fingerprints into drawers* so you only compare the ones already in the same drawer.

My actual configuration

`nepali/scripts/corpuslib/dedup.py`: `DEFAULT_NUM_PERM = 100`, `DEFAULT_BANDS = 20`, `DEFAULT_ROWS = 5`, `DEFAULT_SHINGLE = 5` (characters), Jaccard threshold **0.55**, seed 20260817. Classes and functions: `MinHasher` (with `signature()`), `shingles()`, `build_lsh_buckets()`, `remove_near_duplicates()`, `find_contaminated()`.

Two follow-ups you should expect

"Why 20 bands of 5?" — Bands × rows must equal the number of permutations ($20 \times 5 = 100$). The split sets how sharp the threshold is: more bands is more permissive (catches more, more false positives), fewer bands is stricter. 20×5 pairs with a 0.55 threshold.

"Why threshold 0.55 and not 0.8?" — 0.55 is aggressive: it removes documents that merely overlap substantially. For a web-scraped corpus with heavy boilerplate repetition that is what I wanted; the cost is some false positives, which at 2.2M documents I can afford.

A detail worth volunteering

When a duplicate pair had one copy in the manual lane and one in the downloaded lane — **21,273 pairs** — the resolver keeps the **manual** copy. Otherwise deduplication would quietly eat the manual token share I was required to hit.

3.6 Splits, and why document-level

Short answer — say this

Both corpora are split **by document, never by chunk**. If you split by chunk, two halves of the same article can land on opposite sides, and your test score is partly measuring memorisation rather than generalisation.

	Model H	**Model L**
Ratio	96 / 2 / 2	85 / 7.5 / 7.5
Seed	42	20260817
Method	shuffle then slice	deterministic hash of `doc_id`
Held out *before* quality filtering	no	**yes** (stage S07)
Train tokens	485,280,741	519,906,359
Validation tokens	10,484,640	57,839,396
Test tokens	10,086,647	57,458,578

If the examiner asks for more

Nepali holds out validation and test at stage S07, *before* the quality filters in S05 and S06 run. That means the filters only shape the training pool and cannot make the evaluation sets artificially easy. `split_stats.json` records `zero_doc_id_overlap: true`, and `scripts/check_nepali_hindi_overlap.py` found **0** documents shared between the two languages' corpora.

Common follow-up

"Why different ratios?" — Nepali has far more documents, so 7.5% is still a very large evaluation set (57M tokens) while leaving 520M to train on. Hindi has fewer documents, so 2% is the right trade-off there. I would acknowledge this is an inconsistency between the two pipelines.

3.7 Tokenizers

3.7.1 BPE and Unigram

BPE starts from characters, repeatedly finds the most frequent adjacent pair, and merges it into a new symbol. The vocabulary is the sequence of merges. It is greedy and bottom-up.

Unigram starts from a large candidate vocabulary and *prunes* it: it assumes tokens are drawn independently and iteratively removes the tokens whose removal costs the least likelihood. It is probabilistic and top-down, and at encoding time it picks the highest-probability segmentation.

Why each? I chose them per language from a measured sweep rather than by preference. Unigram won for Hindi, BPE for Nepali, on fertility and unknown rate measured on held-out text.

Tokenization changes the unit of measurement

$$\texttt{"internationalization"} \longrightarrow [\texttt{inter}, \texttt{nation}, \texttt{al}, \texttt{ization}]$$

One word, four tokens. A different tokenizer might give three or six. **This is exactly why perplexity is not comparable across tokenizers** — see Chapter 8.

3.7.2 The measured sweep

Language	Vocab	Chars/token	Fertility	Unk rate	Byte fallback
Hindi	5,000	3.3230	1.4961	0.000000	0.018301
Hindi	**8,000**	**3.5622**	**1.3956**	**0.000000**	**0.019613**
Hindi	10,000	3.6601	1.3583	0.000000	0.020213
Nepali	5,000	3.7882	1.6988	0.002956	0.000000
Nepali	6,000	3.9410	1.6329	0.003075	0.000000
Nepali	8,000	4.1725	1.5424	0.003256	0.000000
Nepali	**10,000**	**4.3410**	**1.4825**	**0.003387**	**0.000000**

Fertility = tokens per word. Lower is better: each token covers more text, so a fixed context window holds more content.

Unknown-token rate = fraction of tokens emitted as `<unk>`. Every one is text the model cannot even represent, let alone predict.

Byte fallback = when the tokenizer meets something outside its vocabulary it encodes the raw UTF-8 bytes instead of emitting `<unk>`. Nothing is ever lost.

Short answer — say this

Fertility keeps improving with vocabulary size, so the choice is a trade-off against embedding parameters. For Hindi, 8,000 → 10,000 buys less fertility than 5,000 → 8,000 did and would cost about 1M parameters out of a 25M budget, so I stopped at 8,000. Nepali took 10,000, the best fertility in its sweep.

The byte-fallback story — your best Phase 1 answer

Nepali's byte-fallback column is `0.000000` at *every* vocabulary size. At the time I read a column of zeros as a good score. **It is not a score — it records that the feature is absent.** That is why Nepali's unknown rate is non-zero and Hindi's is exactly zero.

And it came back in Phase 3: the Nepali vocabulary contains **no Devanagari digit at all**, not one of ०–९, so with no byte fallback every number in my generated reasoning data became `<unk>` — 4.8% of the corpus. I caught it by tokenizing the generated set and checking its unknown rate *before* training, and fixed it by switching Nepali numerals to Arabic digits, which the vocabulary covers completely and which modern Nepali writing uses anyway.

Common follow-up

"Why does the unknown rate go UP with vocabulary size for Nepali?" — Larger vocabularies spend their budget on longer merged units, so rarer characters are less likely to survive as standalone tokens. With byte fallback this would be invisible; without it, it surfaces as `<unk>`.

3.7.3 Why separate tokenizers and no shared vocabulary?

Short answer — say this

The brief requires it — the two models must not share data, tokenizer, vocabulary or weights. But it is also methodologically right: a shared vocabulary would make this a multilingual model, and the point was two independent monolingual models.

If the examiner asks for more

A shared Devanagari vocabulary would have been technically easy since both languages use the same script, which is presumably why the rule exists. It would also have destroyed the comparison — I could no longer attribute any difference to the tokenizer if both models used the same one.

3.8 What I would improve in Phase 1

1. Run MinHash near-duplicate removal and decontamination on Hindi too.
2. Enable byte fallback on the Nepali tokenizer.
3. Use the same manual-share methodology for both — Hindi's is character-weighted, Nepali's token-weighted, so they are not strictly like-for-like.
4. Check how much public text actually exists for a language *before* building a pipeline around it (the Bhojpuri lesson).

Chapter 4

Phase 2 — Architecture, Pretraining, Evaluation

4.1 The architecture

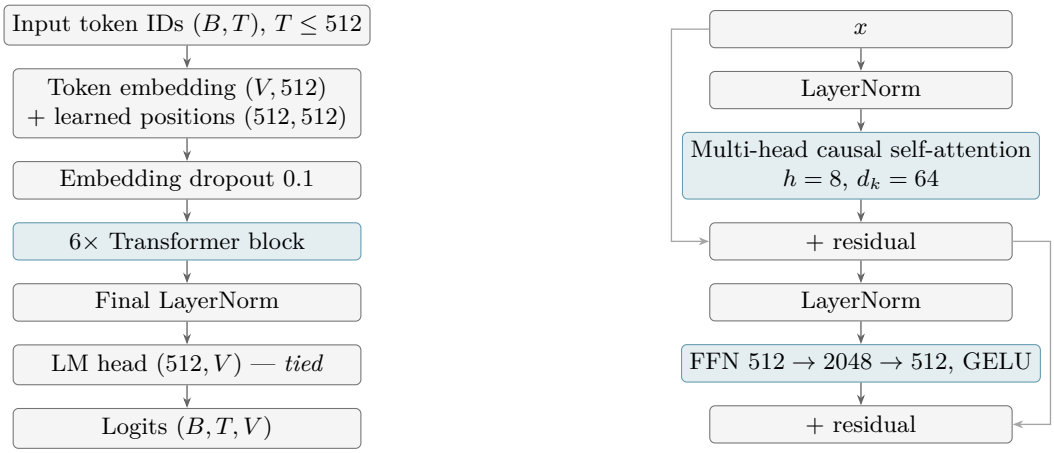


Figure 4.1: Left: the forward pass. Right: one pre-norm block expanded, with the two residual paths drawn in grey. $V = 8{,}000$ for Model H and 10,000 for Model L; everything else is identical between the two models.

Classes: `GPT`, `TransformerBlock`, `MultiHeadCausalSelfAttention`, `FeedForward` and `LayerNorm`, all in `hindi/model/transformer.py` and the mirrored `nepali/model/transformer.py`.

4.1.1 Why decoder-only?

Short answer — say this

The task is causal language modelling — predict the next token given everything before it. A decoder-only stack with causal masking is exactly that. An encoder would let every position see every other position, which makes next-token prediction trivial and useless.

Common follow-up

"Why not encoder-decoder?" — That architecture maps one sequence to another, for translation or summarisation. I have one sequence and one objective.

4.2 Attention

Think of it like this

It is a lookup in a library. Each token writes a **query** ("what am I looking for?"), every token also advertises a **key** ("what am I about?") and carries a **value** ("here is my actual content"). You match your query against all keys to get relevance scores, turn those into weights, and take a weighted blend of the values.

$$\text{Attention}(Q,K,V) \;=\; \text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

Symbol	Shape in my model	Meaning
X	$(B,T,512)$	input to the block
$Q = XW_Q$	$(B,8,T,64)$	queries
$K = XW_K$	$(B,8,T,64)$	keys
$V = XW_V$	$(B,8,T,64)$	values
$QK^\top$	$(B,8,T,T)$	raw relevance scores
d_k	64	per-head dimension
M	$(1,1,T,T)$	additive causal mask
output	$(B,T,512)$	after $\times V$, concat, W_O

The reshape, in words: each projection produces $(B,T,512)$. A `view` turns it into $(B,T,8,64)$ and a `transpose` into $(B,8,T,64)$ — eight independent 64-dimensional heads. After attention, transpose and reshape back to $(B,T,512)$ and apply W_O.

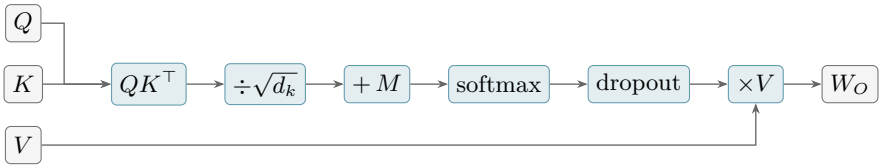


Figure 4.2: The computation, end to end. W_Q, W_K and W_V are kept as three separate `nn.Linear` layers rather than one fused matrix — marginally slower, but the three roles stay visible in the code.

4.2.1 Why divide by $\sqrt{d_k}$?

Short answer — say this

Dot products of longer vectors grow in magnitude. With $d_k = 64$, if the components are roughly independent with unit variance, the dot product has variance about 64 and standard deviation about 8. Feeding numbers that large into softmax saturates it, and the gradient through a saturated softmax is almost zero. Dividing by $\sqrt{64} = 8$ brings the variance back to about 1.

What saturation looks like

$$\text{softmax}([2, 1, 0]) \approx [0.67,\ 0.24,\ 0.09] \quad \text{(informative)}$$

$$\text{softmax}([16, 8, 0]) \approx [0.9997,\ 0.0003,\ \approx 0] \quad \text{(saturated, gradient} \approx 0\text{)}$$

Key point

Without the scaling the model does not fail loudly — it just trains badly, because gradients through attention vanish.

4.3 Causal masking

Simple statement

The model must not be allowed to look at the answer while predicting it.

For the sequence $A\,B\,C\,D$, the allowed pattern is lower triangular:

	key A	**key** B	**key** C	**key** D
query A	yes	—	—	—
query B	yes	yes	—	—
query C	yes	yes	yes	—
query D	yes	yes	yes	yes

and **not** every row full. When predicting what follows C, the model may use A, B and C, but not D — D is the thing being predicted.

If the examiner asks for more

How it is implemented: an upper-triangular mask built with `torch.triu(..., diagonal=1)`, registered as a non-persistent buffer, applied with `masked_fill` with $-\infty$ **before** the softmax.

Why before softmax, and why $-\infty$? Because softmax exponentiates: $e^{-\infty} = 0$ exactly, and softmax then renormalises over the remaining positions so the row still sums to 1. If you masked *after* softmax you would have to renormalise yourself, and the masked probability mass would already have been mixed in.

Why exactly zero and not just small? $\text{softmax}(-\infty)$ is exactly 0, and $0 \times v = 0$. There is no leakage at all, not even at floating-point precision.

How my test verifies it

`test_causal_mask_blocks_the_future()` in `hindi/tests/test_model.py`:

1. Run the model, record the logits.
2. Perturb **every** token after position t.
3. Re-run and measure the largest change in the logits at positions $\leq t$.
4. Result: **exactly 0.000e+00**.
5. The test also asserts the **converse** — that outputs at the perturbed positions *do* change.

Why step 5 matters: without it, a model that ignored its input entirely would also pass. Checking only that nothing leaks is a one-sided test. Both language suites pass 9/9.

4.4 The rest of the block

4.4.1 Pre-norm versus post-norm

Short answer — say this

I used pre-norm — LayerNorm is applied to the input of each sublayer, before attention and before the feed-forward network, rather than after the residual addition.

If the examiner asks for more

With pre-norm the residual path is a clean identity from input to output, so gradients reach early layers without passing through a normalisation at every step. Post-norm needs careful warmup to stay stable in deep stacks. At 6 layers either would probably work; pre-norm is the safer default and is what modern GPT-style models use.

4.4.2 Feed-forward network and GELU

Short answer — say this

Two linear layers, $512 \to 2048 \to 512$, with GELU in between. The inner dimension is $4\times$ the model dimension, the standard ratio.

If the examiner asks for more

Why have an FFN at all? Attention mixes information *across positions* but is linear in the values. The FFN is where per-position non-linear transformation happens. Without it, stacking attention layers would collapse toward a linear operation.

Why GELU over ReLU? It is smooth, so it has a non-zero gradient for small negative inputs instead of ReLU's hard zero, and it is the standard choice in GPT-style models.

4.4.3 Residual connections

Short answer — say this

Each sublayer computes $x + \text{sublayer}(\text{norm}(x))$. The addition gives gradients a direct path back through the network.

If the examiner asks for more

Without them, gradients propagate through every transformation in series and in a 6-layer stack shrink enough that early layers barely learn. Residuals also let a block learn an *adjustment* to its input rather than having to reproduce the whole representation.

4.5 Parameter count — derive this live

Configuration: V = 8,000 (H) or 10,000 (L), $d = 512$, $n_{\text{layer}} = 6$, $n_{\text{head}} = 8$, $d_{ff} = 2048$, block = 512, bias on, weight tying on.

Component	Formula	Model H	Model L
Token embedding	$V \times 512$	4,096,000	5,120,000
Positional embedding	512×512	262,144	262,144
embedding subtotal		*4,358,144*	*5,382,144*
Attention, per block	$4 \times (512^2 + 512)$	1,050,624	1,050,624
FFN, per block	$(512{\cdot}2048{+}2048) + (2048{\cdot}512{+}512)$	2,099,712	2,099,712
LayerNorms, per block	$2 \times (512 + 512)$	2,048	2,048
block total		*3,152,384*	*3,152,384*
6 blocks	$6 \times 3{,}152{,}384$	18,914,304	18,914,304
Final LayerNorm	$512 + 512$	1,024	1,024
Non-embedding		**18,915,328**	**18,915,328**
LM head	tied to token embedding	0	0
Total		**23,273,472**	**24,297,472**

Key point

The non-embedding count is **identical at 18,915,328 for both models**, and that is deliberate — it means nobody can explain a difference in results by saying one model was bigger. The only difference is the embedding table, because the vocabularies differ.

This is also a test

`test_parameter_count_matches_analytic_estimate()` asserts the analytic formula against the loaded model, and `GPT.parameter_count()` returns the breakdown.

4.5.1 Weight tying

Short answer — say this

The output projection shares its matrix with the input token embedding. For Model H that saves $8{,}000 \times 512 = \mathbf{4{,}096{,}000}$ parameters, roughly 15% of the model.

If the examiner asks for more

Both matrices relate the same vocabulary to the same 512-dimensional space — one maps ID → vector, the other vector → score per ID. Two independent representations of the same vocabulary are redundant at this scale, and tying also acts as a regulariser.

Downside? It constrains input and output representations to live in the same space, which at very large scale can cost a little quality. At 23M parameters against a 25M budget, spending 4M on a second copy of the vocabulary was clearly the worse trade. `test_weight_tying_shares_storage()` checks they are the same storage, not merely equal values.

4.5.2 Positional embeddings

Short answer — say this

Learned absolute positional embeddings — a $(512, 512)$ table added to the token embeddings.

If the examiner asks for more

Why positions at all? Self-attention is permutation invariant: shuffle the input tokens and, without position information, the attention output set is unchanged. "Ram beat Shyam" and "Shyam beat Ram" would be identical to the model.

Why learned rather than sinusoidal? The model is trained and evaluated at one fixed context length, so the extrapolation advantage of sinusoidal encodings does not apply, and learned positions let the model allocate resolution where the data actually needs it.

What it costs: 512 is a hard ceiling — there is no embedding row for position 512, so the model cannot run on a longer sequence without retraining the table. `test_sequence_longer_than_block_size_is_rejected()` asserts this fails loudly rather than silently misbehaving. Both alternatives are implemented: `sinusoidal_positional_encoding()` exists, and `pos_encoding: none` is what the bonus ablation uses.

4.6 Training

Optimiser	AdamW, $\beta = (0.9, 0.95)$
Weight decay	0.1, applied only where `param.dim() >= 2`
Peak / min learning rate	$6 \times 10^{-4} \rightarrow 6 \times 10^{-5}$
Schedule	500 warmup steps, then cosine (`CosineWarmupSchedule`)
Gradient clipping	1.0, applied **after** `unscale_`
Micro-batch × accumulation	$32 \times 2 \times 512 =$ **32,768 tokens per step**
Precision	bfloat16
Seed	20260902
Planned steps	15,000
Steps reached	H 14,520 / L 14,440 (11-hour wall-clock guard)
Tokens consumed	H 475,791,360 / L 473,169,920
Hardware	Kaggle Tesla T4, 10.99 h each, ≈12,024 / 11,958 tok s^{-1}

If the examiner asks for more

[Why each of those choices] **AdamW rather than Adam:** it decouples weight decay from the gradient update. In plain Adam, L2 regularisation gets scaled by the adaptive learning rate and is therefore applied inconsistently across parameters.

$\beta_2 = 0.95$ **rather than 0.999:** a shorter second-moment window adapts faster, the common choice for LM pretraining at this scale.

Decay only matrices: LayerNorm gains and biases are not weights in the same sense — decaying them just shrinks them toward zero for no regularisation benefit. The filter is `param.dim() >= 2` in `configure_optimizers()`.

Warmup: at step 0 Adam's moment estimates are unreliable and the parameters are random, so a full-size step can push the model somewhere it takes a long time to recover from.

Cosine decay: a large learning rate early explores, a small one late refines, and cosine is smooth without a hand-tuned staircase.

Gradient accumulation: the 32,768-token effective batch does not fit in T4 memory as one micro-batch. Accumulating 2 micro-batches of 32 decouples the statistical batch size from the memory limit. On a smaller GPU you lower `batch_size` and raise `grad_accum_steps` to keep the product constant.

Clip after unscale: with mixed precision the loss is scaled up to avoid underflow, so gradients are scaled too. Clipping before unscaling would clip the *scaled* gradients and the effective threshold would depend on the scale factor.

4.6.1 Checkpointing and resume

If the examiner asks for more

Why all of it? Restoring only the weights restarts the optimiser's moment estimates and the learning-rate schedule from scratch, which is a different training run. Restoring the RNG states means the data order continues rather than restarting.

Evidence it works: I verified resume across three deliberately killed sessions before starting the real runs, because the whole thing runs on Kaggle and sessions do die.

Why the 11-hour guard? So a dying session could never leave an unusable state. It fired

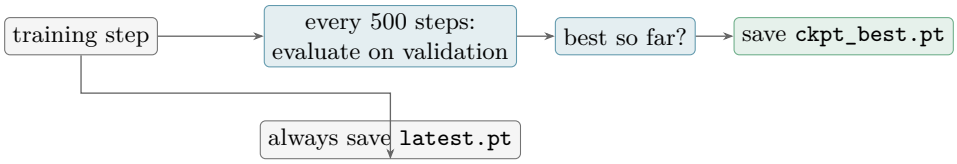


Figure 4.3: Checkpointing. The payload from `_checkpoint_payload()` is model weights, optimizer state, scheduler state, training step, best validation loss, tokens seen, both RNG states and the full config.

at steps 14,520 and 14,440 of a planned 15,000 — costing about 3% of the token budget.

4.7 Intrinsic results

	Model H	**Model L**
Test cross-entropy (nats)	3.3109	3.8433
Test perplexity	**27.41**	46.68
Bits per token	4.7766	5.5447
Bytes per token	8.875	11.441
Test bits-per-byte	0.5382	**0.4846**
Tokens scored	1,999,872	1,999,872
Bytes scored	17,749,798	22,880,653
Best validation loss / ppl	3.3402 / 28.22 @ 14,500	3.5953 / 36.43 @ 14,000

The full perplexity-versus-bits-per-byte argument is Chapter 8 — that is the single most important section in this guide.

4.8 Generation

Setup: 64 held-out prefixes of 64 tokens each, continued for 64 tokens, scored against the true continuations.

Model	**Decoding**	**BLEU-4**	**chrF++**	**ROUGE-L**	**Repetition**	**Distinct-1**	**Distinct-2**
H	greedy	1.96	12.49	0.093	0.598	0.172	0.313
H	$T = 0.5$	1.80	14.33	0.096	0.212	0.275	0.602
H	$T = 1.0$	1.01	**16.29**	**0.100**	0.002	0.520	0.939
H	$T = 1.5$	0.16	14.67	0.053	0.000	0.702	0.992
L	greedy	2.37	14.48	0.081	0.577	0.179	0.263
L	$T = 0.5$	1.88	16.54	**0.092**	0.190	0.345	0.611
L	$T = 1.0$	1.69	**18.05**	0.079	0.001	0.634	0.948
L	$T = 1.5$	0.78	16.13	0.035	0.000	0.793	0.995

Temperature, with numbers

Temperature divides the logits before softmax. Starting from $A = 0.7$, $B = 0.2$, $C = 0.1$:

- $T < 1$ (say 0.5) **sharpens**: the gap widens, A moves toward 1. More confident, more repetitive.

- $T = 1$ leaves the distribution unchanged.
- $T > 1$ (say 1.5) **flattens**: the three move toward equal. More diverse, more errors.
- $T \to 0$ is greedy decoding — always take the argmax.

If the examiner asks for more

Why greedy degenerates. At greedy, 58–60% of generated 4-grams are repeats. That is not a bug in the model — it follows from the rule. Greedy always takes the argmax, so the moment the model re-enters a state it has been in, it produces the same continuation, with no mechanism to escape the loop.

The clean finding. $T = 1.0$ gives the best chrF++ for both models *and* kills repetition. The usual quality–diversity trade-off does not bind anywhere in this range.

Short answer — say this

BLEU counts exact 4-gram overlap with a reference, and an open-ended continuation has no particular reason to reproduce the one true continuation's 4-grams — a perfectly fluent continuation can score near zero. chrF++ is the metric I would defend here, because it works on character n-grams: in morphologically rich Devanagari, changing one suffix destroys a whole-word match but leaves most character n-grams intact. ROUGE-L is computed over whitespace tokens, which under-segment Devanagari, making it the least reliable of the three. I report all three because the specification requires them, and I say which one I trust.

4.9 Attention analysis

Attention entropy measures how spread out a head's attention is: $H = -\sum p \log p$. Low entropy means one key takes almost all the weight (sharp, focused); high entropy means the weight is spread evenly (diffuse).

Mean attention distance measures how far back a head looks, on average, from a query position to the key positions it attends to. Small = local, large = long-range.

	Model H	Model L
Mean entropy (nats)	3.2047	3.0898
Uniform baseline	5.2565	5.2565
Mean distance (tokens)	62.36	54.51
Local-head fraction	0.0625	0.1250

Key point

Why compute the uniform baseline? Because "entropy 3.2" means nothing on its own. Against a uniform baseline of 5.2565 you can say the heads are meaningfully more focused than chance. Without it the number is uninterpretable. (`uniform_baseline_entropy()` in `hindi/eval/attention.py`.)

Layer	Model H (entropy @ distance)	Model L
0	4.16 @ 45.8	3.90 @ 36.0
1	3.09 @ 21.9	3.17 @ 17.1
2	1.49 @ 7.6	1.81 @ 10.6
3	**0.06 @ 1.3**	**0.14 @ 1.1**
4	1.49 @ 22.4	1.32 @ 5.6
5	2.99 @ 151.4	2.48 @ 81.3

Short answer — say this

Both models independently grew a **previous-token head at layer 3** — entropy 0.06 at distance 1.3 for Hindi, 0.14 at distance 1.1 for Nepali. Different languages, different tokenizers, different random seeds, same structure. Attention is diffuse at layer 0, sharpens to layer 3, then broadens again at layer 5.

If the examiner asks for more

Why would a previous-token head emerge? Language is locally predictable — the immediately preceding token is the single most informative context for the next one, especially for morphology and agreement. A head that copies "what came immediately before" is cheap to learn and immediately useful, so gradient descent finds it early.

Say this before they ask

"This is a consistent pattern, not evidence of a functional role. I measured which heads are sharp and how far they look. I did **not** establish what any head contributes to the prediction — that would need ablating a head and re-scoring, which I did not run."

Chapter 5

Phase 3 — Reasoning Finetuning

5.1 Why synthetic data?

Short answer — say this

So that I know the ground-truth answer by construction and can control the train/test relationship exactly. The brief also requires it — the primary finetuning set must not be a downloaded benchmark.

If the examiner asks for more

With a generator I can guarantee that test questions use entity names and sentence templates the model has never seen, and *assert* it. With a downloaded dataset I would have no idea what overlaps what.

5.2 The four task families

All examples below are real rows from `hindi/reasoning/data/test.jsonl` and `nepali/reasoning/data/test.jsonl`.

`compare2` — one-step comparison, 2 candidates

ध्यान दीजिए: प्रवीण वरुण से हल्का है। भारी कौन है? → **वरुण**

(Praveen is lighter than Varun. Who is heavier?) — note the direction flip: "lighter" in the premise, "heavier" in the question.

`numeric2` — explicit quantities, 2 candidates

दिया गया है कि चंदा का वज़न ६७ किलो है और मालती का वज़न ४८ किलो है। हल्की कौन है? → **मालती**

`numeric3` — explicit quantities, 3 candidates

यहाँ प्रवीण की उम्र ५९ साल, नवीन की उम्र ६७ साल, तथा अरुण की उम्र २४ साल है। सबसे बड़ा कौन है? → **नवीन**

`transitive3` — two-hop chain, 3 candidates

शारदा सविता से भारी है। इसी तरह सविता मालती से भारी है। सबसे भारी कौन है? → **शारदा**

(Sharda > Savita, Savita > Malti. Who is heaviest?)

Nepali — note the Arabic digits

दइिएको छ कि रिबनिको उमेर 22 वर्ष र भोजराजको उमेर 49 वर्ष छ। जेठो को हो? → **भोजराज**

Hindi uses Devanagari digits (५९, ६७); Nepali uses Arabic digits. That asymmetry is the byte-fallback fix described in Chapter 3.

5.2.1 Why `transitive3` is the real test

Premise 1:	$A > B$	
Premise 2:	$B > C$	
Question:	who is largest?	Answer: A

If the examiner asks for more

What information is needed? You must chain the two inequalities — neither premise alone identifies A as the maximum.

Why is this transitive reasoning? It requires the inference $A > B \wedge B > C \Longrightarrow A > C$, which is stated nowhere in the prompt.

What would a shortcut-based model do? Notice that B **is the only name mentioned twice** and pick it on salience. That is exactly what my models do — see Section 5.6.

5.3 Dataset construction and leakage control

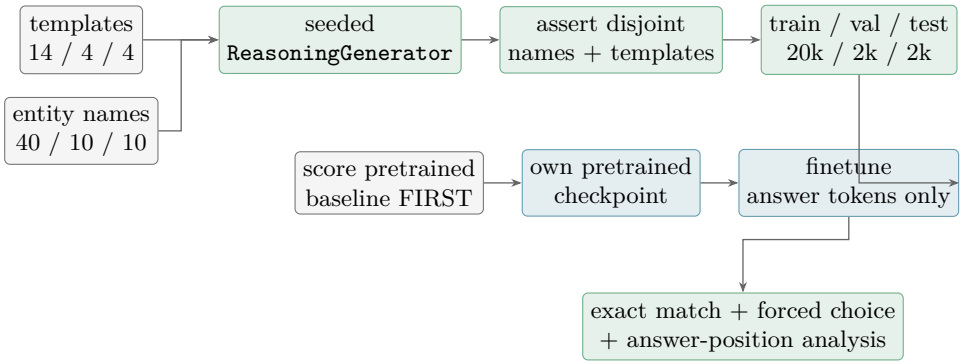


Figure 5.1: Generation and finetuning. Scoring the pretrained baseline *before* finetuning is a sequencing constraint — once the weights change that number is unrecoverable without retraining.

Short answer — say this

Leakage is controlled on two axes at once. The 40/10/10 entity names and the 14/4/4 sentence templates are both disjoint across splits, so a test question is about people the model has never seen, phrased in a way it has never seen.

If the examiner asks for more

Both properties are **asserted inside `main()` of `generate.py`**, not assumed, and recorded in `dataset_stats.json` as `entity_names_disjoint_across_splits: true` and `templates_disjoint_across_splits: true`. The Hindi and Nepali sets share no sentence, no entity name and no template.

"Why hold out both?" Holding out only names would let the model pass by memorising which template implies which answer position. Holding out only templates would let it memorise per-name answer priors. You need both axes closed.

5.3.1 The gender-agreement leak

A defect I found and fixed — good to volunteer

Problem: Hindi adjectives inflect for gender, so a question like "who is smallest?" asked about one woman and two men has only one grammatically well-formed answer — solvable without comparing anything.

Detection: inspecting generated examples for shortcuts. **181 of 400** sampled examples were solvable that way.

Why: the generator drew entity names independently of gender, and the adjective had to agree with *some* gender.

Fix: `inflect()` plus drawing all entities in an example from one gender, so agreement carries no information about the answer.

Lesson: in an inflected language, grammar itself can be a leak channel.

5.4 Finetuning protocol

Starting point	each model's **own** Phase 2 checkpoint
Tokenizer	held fixed from Phase 1
Loss	answer tokens only (`ignore_index=-1` over the prompt)
Batching	grouped by length, **not** padded
Epochs / batch	2 / 32
Peak / min LR	6×10^{-5} / 6×10^{-6}
Warmup	100 steps
Evaluation interval	every **20** steps
Seed	20260914
Precision	float16 with a `GradScaler`

If the examiner asks for more

[Why each choice] **Answer-token-only loss:** if loss covered the whole sequence, most of it would be question text and the model would be rewarded for memorising phrasing rather than producing the right answer. Masking the prompt means the gradient only ever comes from the answer.

Length-grouped batches — necessary, not cosmetic: with learned absolute positional embeddings, left padding would shift the position index of every real token. The same content

would sit at position 3 in one batch and 7 in another, and the model would see a different positional embedding for it. Grouping by length avoids padding entirely.

Learning rate one tenth of pretraining: the model already holds a language model worth keeping. A pretraining-sized step overwrites it before the task is learned; the usual failure mode is a model that answers in the right format but has forgotten how to read the question.

float16 rather than bfloat16: the Kaggle T4 is compute capability 7.5 and has no bfloat16 support, so the pretraining config could not be reused. float16 needs a `GradScaler` to avoid gradient underflow.

A guard worth mentioning

The finetuning script **asserts** that the tokenizer vocabulary size matches the checkpoint, rather than trusting the config file. That mismatch is exactly the kind you do not notice until the results are garbage.

5.5 Exact match versus forced choice

Exact match	Forced choice
Greedy-decode an answer. Does the string equal the gold answer?	Take only the entity names the question actually contains. Score each one's length-normalised mean log-probability. Does the highest-scoring one match?
Honest end-to-end measure — but pinned at 0 before finetuning, so it cannot separate "learned nothing" from "knows it but cannot phrase it".	Removes the formatting problem entirely. Chance rate computed per example from the number of candidates.

Key point

The chance rate. `compare2` and `numeric2` have 2 candidates (chance 50%); `numeric3` and `transitive3` have 3 (chance 33.3%). Averaged over the equal-sized families: $(0.5 + 0.5 + \frac{1}{3} + \frac{1}{3})/4 = \mathbf{0.4167}$. It is computed per example from the actual candidate count, not assumed.

	H pretrained	H finetuned	L pretrained	L finetuned
Exact match	0.00%	**35.15%**	0.00%	**29.45%**
Forced choice	39.80%	36.35%	38.80%	39.30%
Chance rate	41.67%	41.67%	41.67%	41.67%
Answer-token loss	5.2439	0.9408	5.5367	1.4351
Answer-token perplexity	189.40	2.56	253.84	4.20

The answer to give — observation, interpretation, limit

Observation: exact match rose from zero to 35.15% and 29.45%, and answer-token loss fell by more than four nats in both models.

Interpretation: finetuning taught the models to emit a candidate name in the expected

position and format. The loss collapse is consistent with that and with nothing stronger.

What it does not show: under forced choice both models sit at or below the 41.67% chance rate, and per task the scores land close to their individual chance rates. The result does not establish that either model learned the comparison operation.

Task	Chance	H exact	L exact	H forced	L forced
`compare2`	50.0%	47.40%	40.20%	50.40%	49.40%
`numeric2`	50.0%	47.40%	38.00%	47.00%	51.80%
`numeric3`	33.3%	33.40%	26.20%	33.20%	32.40%
`transitive3`	33.3%	12.40%	13.40%	**14.80%**	**23.60%**

Three families sit on their chance line. The two-hop family falls **below** it.

5.6 Error analysis

`transitive3`, finetuned	**answered** A (first)	**answered** B (middle)	**answered** C (last)	**off-prompt**
Gold distribution	47.4%	**0.0%**	52.6%	—
Model H	**0.0%**	**72.6%**	25.4%	2.0%
Model L	3.2%	41.8%	26.2%	28.8%

The argument, in one breath

The chain is stated as A rel B, B rel C, so the extremes are always A and C and the middle entity is **never** correct. Model H nevertheless picks the middle entity 72.6% of the time and the first entity in **none**. B is the only name mentioned twice. So the model is selecting on surface salience, not following the inequality — and that is precisely why this family scores *below* chance, because uniform guessing would at least hit A and C sometimes.

One-hop recency bias, Model H: on `compare2` it answers the second-named entity 62.0% of the time against a 48/52 gold split, and on `numeric2` 65.0% against a 50/50 split — roughly two to one toward the most recently mentioned name.

Answers with a name not in the prompt	Model H	Model L
`compare2`	7.2%	19.2%
`numeric2`	0.0%	25.0%
`numeric3`	1.0%	21.2%
`transitive3`	2.0%	28.8%

Key point

Model L emits memorised *training* names (सरिजना, नरिु) instead of a candidate in front of it. Before finetuning both models do this essentially 100% of the time. So the resource tier shows up not as reasoning accuracy — both are at chance there — but as **groundedness**. The better-pretrained model is better at *using its input*, which is a weaker and far more defensible claim than "better reasoning".

5.7 Overfitting and checkpoint selection

Short answer — say this

Validation loss bottoms out at **step 60** and rises afterwards while training loss keeps falling to 0.4441 (H) and 0.6048 (L), reaching validation 3.5188 and 3.5911 by step 1,250. I save on best validation loss, so the evaluated checkpoints are step 60 — the overfitting visible in the curve never reached my reported numbers.

If the examiner asks for more

Why did it happen? With only 40 entity names and 14 templates, memorising which answer tends to be right is easier than learning to read the question. The training distribution is too narrow in two independent ways: too few entities, and every example of a template is identical apart from slot fillers, so the surface form carries almost no entropy.

The Run A / Run B story — volunteer this

My first finetuning run evaluated every **100** steps and selected its best checkpoint at step 100 — which is exactly where warmup ends and the learning rate first reaches its peak — with validation loss rising monotonically after that. The grid was too coarse to locate the real optimum.

I moved to a **20**-step grid with nothing else changed, and Model L went from 14.90% to 29.45% exact match, Model H from 32.70% to 35.15%. The superseded run is kept in `report/phase3/run_a_40names/` rather than deleted.

That is a one-variable change with a measured effect, and keeping the old run is good practice.

5.8 Attention before and after finetuning

Measured on the same held-out reasoning prompts, all of identical token length, so that neither prompt content nor sequence length could masquerade as a finetuning effect.

Model	Checkpoint	Mean entropy	Mean distance	Uniform baseline
H	pretrained (step 14,500)	1.9102	10.303	2.9545 (length 46)
H	finetuned (step 60)	1.8691	10.360	
L	pretrained (step 14,000)	1.8327	9.415	2.8061 (length 39)
L	finetuned (step 60)	1.7833	10.157	

Observation: attention became slightly sharper *and* slightly longer-range at the same time in both models, most visibly in Model L where mean distance rises 0.74 positions on a 39-token prompt. The largest per-head shifts concentrate in the middle layers — where Phase 2 found the previous-token heads.

Interpretation: consistent with a shift toward attending to the entity names scattered across the prompt rather than to local context.

Limitation — state it yourself

"This identifies which heads changed, not what they compute. Showing that these heads perform the comparison would require ablating a head and re-scoring, which I did not run."

Chapter 6

Cross-Phase Questions

Q: How did tokenizer design affect Phase 2?

Directly and decisively. Model L's BPE at 10,000 produces 11.44 bytes per token against Model H's 8.88. That single property is large enough to reverse the ranking between perplexity and bits-per-byte. Any cross-language perplexity comparison in this project is a statement about tokenizers at least as much as about models.

Q: How did Phase 1 data quality affect Phase 2?

Model L's corpus is nominally larger (520M vs 485M train tokens) but 91.6% of its manual lane is a single news source, while Model H draws on five distinct manual sources and five downloaded partitions. Model L saw *more* tokens of *narrower* text. That is a plausible contributor to its higher perplexity — though I cannot cleanly separate it from tokenizer effects.

Q: Which Phase 1 decision most affected later phases?

The missing byte fallback on the Nepali tokenizer. In Phase 1 it appeared only as a column of zeros I misread as a good score. In Phase 2 it surfaced as a 0.17–0.34% unknown-token rate. In Phase 3 it became outright corruption — no Devanagari digit in the vocabulary, so 4.8% of the generated reasoning corpus became `<unk>`. One configuration flag, invisible for two phases, then responsible for a data defect.

Q: Why might a model with better language modelling still fail reasoning?

Because next-token prediction and relational inference are different capabilities. Both my models are at chance on the comparison itself despite being usable language models. What better pretraining bought was groundedness — Model H answers from the given candidates 93–100% of the time against Model L's 71–81% — not reasoning.

Q: How did Phase 2 attention analysis help Phase 3?

It gave me a baseline. Phase 2 located previous-token heads at layer 3; Phase 3 found the largest finetuning-induced shifts in those same middle layers. Without the Phase 2 measurement, the Phase 3 deltas would have had nothing to be relative to.

Q: What evidence supports your main conclusion, and what does not?

Supports: that most of the H/L perplexity gap is tokenizer granularity — the bytes-per-token measurement (8.88 vs 11.44) is large enough to flip the ordering, and the flip is exactly what bits-per-byte shows.

Does not: bits-per-byte is tokenizer-neutral but **not corpus-neutral**. The two models are

scored on different text of different intrinsic difficulty, so I cannot conclude Model L is the better language model — only that the perplexity gap overstates the difference. Separating those would need a parallel evaluation set, which I do not have.

Q: Strongest result? Weakest? Most surprising? What next?

Strongest: the perplexity/bits-per-byte reversal — clean, mechanistically explained, derivable live.

Weakest: the reasoning results. Both models are at chance, and the training distribution was too narrow to give finetuning a fair chance.

Most surprising: that both models independently grew a near-deterministic previous-token head at layer 3, with different languages, tokenizers and seeds.

Next experiment: head ablation — disable the heads that moved most during finetuning and re-score the reasoning test set. That converts the attention analysis from correlational to causal, which is the main thing it currently lacks.

Chapter 7

Code-Level Questions

7.1 Where everything lives

Area	File	Key names
Cleaning	scripts/clean_lib.py	normalize_text(), devanagari_ratio(), malformed_ratio(), is_too_short(), langdetect_verdict(), dedup_key(), strip_latin_tokens()
Hindi pipeline	hindi/data/pipeline/	download_hindi.py, ingest_manual.py, ingest_wikipedia_dump.py, clean_hindi.py, split_hindi.py
Nepali pipeline	nepali/scripts/	s00_doctor.py … s14_corpus_stats.py, run_pipeline.py
Nepali dedup	corpuslib/dedup.py	MinHasher, shingles(), build_lsh_buckets(), remove_near_duplicates(), find_contaminated()
Nepali language ID	corpuslib/langid.py	LanguageIdentifier, self_test(), classify()
Packing	hindi/data/pack_corpus.py	CorpusPacker.pack_split()
Model	hindi/model/transformer.py	GPT, TransformerBlock, MultiHeadCausalSelfAttention, FeedForward, sinusoidal_positional_encoding(), generate(), parameter_count(), configure_optimizers()
Training	hindi/train/train.py	Trainer, CosineWarmupSchedule, _checkpoint_payload(), _maybe_resume(), evaluate()
Evaluation	hindi/eval/evaluate.py	Evaluator, perplexity_and_bpb(), generation_suite(), attention_report(), rouge_l(), diversity_metrics(), overlap_metrics()
Attention	hindi/eval/attention.py	collect_attention(), attention_entropy(), mean_attention_distance(), summarise_model_attention(), uniform_baseline_entropy(), head_classification()

Area	File	Key names
Reasoning data	`hindi/reasoning/generate.py`	`ReasoningGenerator`, `compare2()`, `transitive3()`, `numeric2()`, `numeric3()`, `inflect()`, `to_devanagari()`
Reasoning dataset	`hindi/reasoning/dataset.py`	`ReasoningDataset`, `batches()`, `prompt_batches()`, `sequence_stats()`
Finetuning	`hindi/reasoning/finetune.py`	`FineTuner`, `FinetuneConfig`
Reasoning eval	`evaluate_reasoning.py`	`ReasoningEvaluator`, `greedy_answers()`, `forced_choice()`, `answer_loss()`, `position_analysis()`, `breakdown()`
Attention compare	`attention_compare.py`	`compare()`, `per_head_table()`, `reasoning_prompt_batch()`
Tests	`hindi/tests/test_model.py`	`test_causal_mask_blocks_the_future()`, `test_parameter_count_matches_analytic_estimate()`, `test_weight_tying_shares_storage()`, `test_no_prebuilt_attention_modules_used()`

7.2 Likely code questions

Q: What does `dedup_key()` do, and what breaks without it?

It produces a SHA-1 hash of the text after collapsing whitespace, stripping punctuation and case-folding, so formatting variants of the same document collide. Without it, exact duplicates survive into training, inflating apparent corpus size and risking train/test overlap. Its own docstring is explicit that it does **not** catch fuzzy or paraphrase similarity.

Q: What does `test_no_prebuilt_attention_modules_used()` check?

It walks the module tree and fails if `nn.Transformer*`, `nn.MultiheadAttention` or any prebuilt attention block appears. It exists so the hard constraint cannot be broken by a later edit — I cannot accidentally violate the rule.

Q: In `MultiHeadCausalSelfAttention.forward()`, what are the tensor shapes?

Input $(B, T, 512)$. Each of the three projections gives $(B, T, 512)$, a `view` makes it $(B, T, 8, 64)$ and a `transpose` makes it $(B, 8, T, 64)$. Scores $QK^\top$ are $(B, 8, T, T)$. After masking, softmax, dropout and $\times V$ we get $(B, 8, T, 64)$, then transpose and reshape back to $(B, T, 512)$ and apply W_O.

Q: Where is dropout applied?

Four places: attention probabilities, attention output, feed-forward output, and the embedding sum. Dropout on the attention probabilities is the one people forget — it stops the model relying on one specific key.

Q: What does `forced_choice()` compute?

For each test example it scores every candidate entity name from the question by its length-normalised mean log-probability under the model, and picks the highest. Length normalisation matters because otherwise longer names are systematically penalised.

Q: Why does `ReasoningDataset.batches()` group by length?

Because padding would shift position indices under learned absolute positional embeddings — the same token would receive a different positional embedding depending on how much padding preceded it.

Chapter 8

Results and Metrics

8.1 Perplexity versus bits-per-byte

Key point

This is the single most important section in the guide. Be able to derive every line of it on paper.

$$\begin{aligned}
\text{Cross-entropy (nats/token):} \quad & L = -\frac{1}{N}\sum_{t} \log p(x_t \mid x_{<t}) \\
\text{Perplexity:} \quad & \text{PPL} = e^{L} \\
\text{Bits per token:} \quad & b = L / \ln 2 \\
\text{Bits per byte:} \quad & \text{BPB} = b \,/\, (\text{bytes per token})
\end{aligned}$$

Derive it live, with my numbers

Model H

$$\begin{aligned}
L &= 3.3109 \text{ nats} \\
\text{PPL} &= e^{3.3109} = 27.41 \\
b &= 3.3109/0.6931 = 4.7766 \text{ bits/token} \\
\text{bytes/token} &= 17{,}749{,}798/1{,}999{,}872 = 8.875 \\
\text{BPB} &= 4.7766/8.875 = 0.5382
\end{aligned}$$

Model L

$$\begin{aligned}
L &= 3.8433 \text{ nats} \\
\text{PPL} &= e^{3.8433} = 46.68 \\
b &= 3.8433/0.6931 = 5.5447 \text{ bits/token} \\
\text{bytes/token} &= 22{,}880{,}653/1{,}999{,}872 = 11.441 \\
\text{BPB} &= 5.5447/11.441 = 0.4846
\end{aligned}$$

The story

Perplexity says Model H is 41% better. Bits-per-byte says Model L is slightly better. Both are correct, because they measure per different units. Perplexity is per *token*, and a token is not the same amount of text in the two models — Model L's BPE holds 11.44 bytes where Model H's Unigram holds 8.88. Model L's tokens are individually harder to predict, but each

one carries about 29% more text. Dividing by bytes takes the tokenizer out of the comparison, and when you do, the ordering reverses.

Q: Why is the model with worse perplexity not necessarily worse?

Because perplexity is per token and the token is defined by the tokenizer, not by the language. A coarser tokenizer makes each prediction harder — more options, more content per step — so it inflates perplexity even if the model is representing the text just as well. You are comparing two different measuring sticks.

Q: Why did the ranking change?

Because bytes-per-token differs by 29% between the two models, which is enough to more than offset a 16% difference in bits per token.

Q: Does bits-per-byte completely solve the comparison problem?

No — and this is the answer that earns credit. BPB is tokenizer-neutral but **not corpus-neutral**. The two models are evaluated on different text of different intrinsic difficulty: Model L's corpus is nominally larger but 91.6% of its manual lane is one news source, so it may simply be more predictable text. Separating "better model" from "easier corpus" would need a parallel or translated evaluation set, which I do not have.

Q: So which model is better?

"On their own test sets, the honest answer is that I cannot rank them cleanly. I can say the perplexity gap overstates the difference, and I can say why."

8.2 Numbers that prove things

- Causal-mask leakage: **exactly 0.000e+00**.
- Non-embedding parameters: **18,915,328 for both models** — capacity controlled.
- Cross-corpus document overlap: **0**.
- Nepali split document-id overlap: **0**.
- Generated reasoning data unknown-token rate after the digit fix: **exactly 0**.

Chapter 9

"Why Did You Make This Choice?"

Question	Best short answer	Deeper / follow-up
Why this tokenizer per language?	I ran a sweep and measured fertility and unknown rate on held-out text; Unigram won for Hindi, BPE for Nepali.	Full sweep in `measured_tokenizer_metrics.json`. "Would BPE have worked for Hindi?" — probably; I chose on measurement, not principle.
Why vocab 8,000 for Hindi?	Fertility keeps improving but 8k→10k buys less than 5k→8k and costs ≈1M parameters out of 25M.	Diminishing returns against a hard parameter budget.
Why 6 layers?	Each block is 3.15M parameters, so 6 plus embeddings is 23.27M; 7 would overshoot to 26.4M.	I preferred depth over width because attention needs several rounds of composition to build multi-hop dependencies.
Why context 512?	It keeps the $T \times T$ attention matrix at 262k entries per head, which makes a 500M-token budget affordable in a Kaggle session.	Attention is $O(T^2)$; doubling context quadruples the cost. At inference, learned absolute positions are the hard limit.
Why causal attention?	The objective is next-token prediction; without the mask the model could see the token it is predicting.	And I verified it empirically, not just structurally.
Why AdamW?	It decouples weight decay from the adaptive update, so regularisation is applied consistently.	In plain Adam, L2 gets scaled by the per-parameter learning rate.
Why LR 6×10^{-4}?	Standard for this model scale, paired with 500 warmup steps and cosine decay.	"Did you tune it?" — No. I spent the compute budget on the two full runs. Say so honestly.
Why gradient accumulation?	The 32,768-token effective batch does not fit in T4 memory as one micro-batch.	$32 \times 2 \times 512$; lower batch size and raise accumulation on a smaller GPU.
Why weight tying?	Saves 4,096,000 parameters (≈15%) and the two matrices describe the same vocabulary in the same space.	Also acts as a regulariser. Downside: constrains input and output to share a space.

Question	**Best short answer**	**Deeper / follow-up**
Why synthetic reasoning data?	So I know the ground truth by construction and can guarantee disjoint names and templates.	The brief also forbids a downloaded benchmark as the primary set.
Why forced choice?	Exact match is pinned at zero before finetuning and cannot separate "learned nothing" from "cannot phrase it".	It is also the less flattering number, which is why it is the informative one.
Why is attention analysis not enough?	It is correlational — it shows which heads are sharp or changed, not what they contribute.	A causal claim needs head ablation and re-scoring.
Why document-level split?	So two halves of the same article cannot land on opposite sides and turn memorisation into test accuracy.	Nepali also holds out before quality filtering.
Why remove duplicates?	They inflate apparent corpus size, overweight repeated text, and can leak into evaluation.	Be honest about the H/L asymmetry.
Why evaluate every 20 steps?	Run A's 100-step grid selected its best checkpoint at step 100, exactly where warmup ends — too coarse.	Changing only that raised Model L from 14.90% to 29.45%.

Chapter 10

What Went Wrong — Limitations and Lessons

Each of these is **Problem → Detection → Fix → Lesson**.

1. **Nepali language ID silently deleted the corpus.** The `fasttext` wheel calls `np.array(probs, copy=False)`, which raises on NumPy ≥2; every prediction threw and the caller swallowed it as "not our language". Detected because the surviving fraction was 5.85%, exactly the one source configured to skip language ID. Fixed by calling the raw binding, adding a startup self-test and counting errors separately from rejections. *Lesson: a bare* `except Exception` *around a classifier turns a library bug into silent data loss.*
2. **Checkpoint-selection grid too coarse.** 100-step grid selected step 100, exactly where warmup ends. Detected by validation loss rising monotonically from the first eval point. Fixed with a 20-step grid: Model L 14.90% → 29.45%, Model H 32.70% → 35.15%.
3. **Gender-agreement leak.** Adjective inflection made 181/400 examples solvable without comparing. Detected by manual inspection. Fixed by drawing all entities from one gender. *Lesson: in an inflected language, grammar itself can be a leak channel.*
4. **Nepali tokenizer has no byte fallback.** No Devanagari digit in the vocabulary; 4.8% of generated reasoning data became `<unk>`. Detected by checking the unknown rate *before* training. Fixed by switching to Arabic digits. *Lesson: a column of zeros is not automatically a good score — check whether it means "none" or "absent".*
5. **Neither pretraining run converged.** Both hit the 11-hour guard at ≈97% of the planned token budget with validation loss still falling. Perplexities are upper bounds. Mitigated because both stopped under the same rule with near-identical budgets, so the *comparison* is unaffected.
6. **Hindi deduplication is weaker than Nepali's.** No near-duplicate removal, no decontamination. An unquantified confound in the direction that flatters Model H.
7. **The reasoning training distribution is too narrow.** 40 names and 14 templates, every example of a template identical apart from slot fillers. Both models overfit by step 60. Fixes: 200+ entities and a per-template variation layer (synonym substitution, clause reordering, optional discourse particles). The generator has no such layer today.
8. **Attention analysis is correlational.** No head-ablation probe was run.
9. **Manual-share methodology differs between languages.** Hindi's is character-weighted over the cleaned corpus, Nepali's token-weighted — not strictly like-for-like.
10. **Evaluation subsampled.** Intrinsic metrics use 2M-token subsets, not the full 10.1M/57.7M test sets — but with an identical budget for both models.

Chapter 11

Trap and Follow-Up Questions

Q: Your perplexity is lower, so Model H is definitely the better model, right?

No. Perplexity is per token, and the two tokenizers produce different-sized tokens — 8.88 against 11.44 bytes. On bits-per-byte the ordering actually reverses, 0.5382 against 0.4846. The honest statement is that the perplexity gap overstates the difference, and that I cannot cleanly rank the two because they are scored on different corpora.

Q: Your attention analysis shows the model is reasoning, correct?

No. It shows which heads are sharp, how far they look, and which ones changed during finetuning. It does not show what any head contributes to the prediction. And separately, my reasoning metrics say the models are at chance, so there is no reasoning for attention to be evidence of.

Q: Exact match went from 0% to 35%, so finetuning taught reasoning?

It taught the answer *format*. Under forced choice — restricting the model to the names already in the question — both models sit at or below the 41.67% chance rate. Per task they land on their individual chance lines. The one family that leaves chance leaves it downward.

Q: Why not just use BLEU for generation?

Because BLEU counts exact 4-gram overlap with one reference, and open-ended continuation has many valid outputs. My BLEU is near the floor at every setting while chrF++ varies meaningfully and peaks at $T = 1.0$. In Devanagari, one changed suffix destroys a whole-word match but leaves most character n-grams intact, so chrF++ is the better instrument.

Q: Why not train longer?

I would have liked to — both validation curves were still falling. The constraint was the Kaggle GPU quota; an 11-hour wall-clock guard stopped each run so a dying session could never leave an unusable state. I report the runs as-run rather than implying convergence.

Q: Why not use a bigger model?

The brief specifies approximately 25M parameters. Within that, I chose 6 layers over a wider 4-layer model, because a seventh layer would have overshot to 26.4M.

Q: Why not one tokenizer for both languages?

The brief forbids sharing vocabulary, and it would have made this a multilingual model rather than two independent monolingual ones. It would also have destroyed the comparison — I could not attribute any difference to the tokenizer if both used the same one. It would have been technically easy since both use Devanagari, which is presumably why the rule exists.

Q: Why not split randomly at the chunk level? It is simpler.

Because two halves of the same article could land on opposite sides, and the test score would partly measure memorisation rather than generalisation.

Q: Why do you need a validation set at all?

To select the checkpoint without touching test. In Phase 3 this was decisive: validation loss bottoms at step 60 and rises afterwards, so the final checkpoint is badly overfit. Without validation-based selection my reported numbers would have come from a memorised model.

Q: Why does a lower loss not always mean better reasoning?

Because loss here is answer-token cross-entropy, and it falls if the model simply learns that the answer is always one of the two or three names in a fixed position. My answer-token loss fell from 5.2439 to 0.9408 while forced-choice accuracy went *down*, from 39.80% to 36.35%. That is the cleanest demonstration in the project that loss and capability can move in opposite directions.

Q: Your two models have different parameter counts — isn't that unfair?

The difference is entirely the embedding table, because the vocabularies differ (8,000 vs 10,000). The **non-embedding** count is identical at 18,915,328. The compute spent on actual transformation is the same.

Q: Could your Hindi results be inflated by train/test leakage?

Possibly, and I would rather say so. Hindi deduplicates only on exact SHA-1 and has no decontamination stage, while Nepali has both. The direction of that bias flatters Hindi. I did not quantify it; fixing it would mean running the Nepali MinHash decontamination over the Hindi corpus.

Q: Isn't your Bhojpuri work wasted?

It is a documented negative result. I built the full pipeline, it passed validation, and public text ran out at 84,217,032 tokens — 16.8% of target. The lesson I took is to measure how much text exists for a language before building a pipeline around it.

Chapter 12

Rapid-Fire

One or two sentences each. Cover the right-hand column and test yourself.

Question	**Answer**
Model H language?	Hindi, higher-resource.
Model L language?	Nepali, from the allowed list.
Why the same script?	To hold orthography constant so differences come from data and tokenizer.
Hindi corpus size?	505,852,028 tokens, 704,101 documents.
Nepali corpus size?	635,204,333 tokens, 1,918,265 documents.
Manual share?	30.66% Hindi, 26.41% Nepali; floor was 20%.
Hindi tokenizer?	SentencePiece Unigram, vocabulary 8,000.
Nepali tokenizer?	SentencePiece BPE, vocabulary 10,000.
Hindi fertility?	1.3956 tokens per word.
Nepali unknown rate?	0.003387 — non-zero because there is no byte fallback.
Hindi unknown rate?	Exactly 0.000000, because byte fallback is on.
Split ratios?	Hindi 96/2/2, Nepali 85/7.5/7.5, both by document.
Hindi dedup?	SHA-1 exact only — 2,983 removed.
Nepali dedup?	MinHash + LSH — 2,350 exact, 49,734 near.
MinHash config?	100 permutations, 20 bands × 5 rows, 5-char shingles, Jaccard 0.55.
Architecture?	6 layers, 8 heads, $d = 512$, $d_{ff} = 2048$, context 512, pre-norm, GELU.
Parameters?	23,273,472 (H) and 24,297,472 (L).
Non-embedding parameters?	18,915,328 — identical for both.
Positional encoding?	Learned absolute; sinusoidal and none also implemented.
Weight tying?	Yes — saves 4,096,000 parameters for Model H.
Why $\sqrt{d_k}$?	$d_k = 64$, so dot products have std ≈8; without scaling softmax saturates and gradients vanish.
Causal-mask leakage?	Exactly 0.000e+00, verified by a test that also checks the converse.
Optimiser?	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1 on matrices only.
Learning rate?	6×10^{-4} peak, 500 warmup, cosine to 6×10^{-5}.
Effective batch?	$32 \times 2 \times 512 = 32{,}768$ tokens per step.
Training tokens?	475,791,360 (H) and 473,169,920 (L).
Hardware and time?	Kaggle Tesla T4, 10.99 hours each.
Test perplexity?	27.41 (H), 46.68 (L).
Test bits-per-byte?	0.5382 (H), 0.4846 (L) — the ranking reverses.

Question	**Answer**
Why does it reverse?	Bytes per token: 8.875 versus 11.441.
Best decoding setting?	$T = 1.0$ — best chrF++ (16.29 / 18.05) and near-zero repetition.
Greedy repetition rate?	0.598 (H), 0.577 (L).
Most trustworthy generation metric?	chrF++, because character n-grams survive morphology.
Mean attention entropy?	3.2047 (H), 3.0898 (L) against a uniform baseline of 5.2565.
Most interesting head?	Layer 3 previous-token head in *both* models — 0.06 @ 1.3 and 0.14 @ 1.1.
Reasoning dataset size?	20,000 / 2,000 / 2,000 per language, four task families.
Leakage control?	40/10/10 names and 14/4/4 templates, both disjoint, asserted in the generator.
Finetuning LR?	6×10^{-5}, one tenth of pretraining.
Loss masking?	Answer tokens only, `ignore_index=-1` over the prompt.
Why length-grouped batches?	Padding would shift position indices under learned absolute positions.
Exact match after finetuning?	35.15% (H), 29.45% (L), from 0.00%.
Forced choice?	36.35% (H), 39.30% (L) — chance is 41.67%.
What does that mean?	Finetuning taught the answer format, not the comparison.
Worst task?	`transitive3` — 12.40% and 13.40% exact match, below chance.
Why below chance?	The model picks the twice-mentioned middle entity 72.6% of the time, and the middle entity is never correct.
Off-prompt answers?	0.0–7.2% for H, 19.2–28.8% for L.
What does that show?	Groundedness is the real resource-tier difference, not reasoning.
Best finetuning checkpoint?	Step 60, by validation loss; validation rises after that.
Attention after finetuning?	Slightly sharper and slightly longer-range; biggest shifts in middle layers.
Can you claim causality there?	No — that needs head ablation, which I did not run.

Chapter 13

The Final One-Minute Conclusion

Closing statement

If I had to summarise the project in one minute: I built two independent 23-million-parameter Transformers from scratch, one Hindi and one Nepali, with separate corpora, separate tokenizers and separate weights, and I held the script and the architecture constant so the comparison between them would actually mean something.

The two results I would stand behind are both cases where a headline number was misleading. Hindi's 41% perplexity advantage mostly disappears under bits-per-byte, because the gap is largely tokenizer granularity rather than model quality. And finetuning raised exact match from 0% to 35%, but forced-choice accuracy stayed at chance, so the models learned the answer format rather than the comparison — the two-hop task actually falls below chance because the model picks the name that gets mentioned twice.

What the resource tier really bought was not reasoning but groundedness: Model H answers from the candidates actually in its prompt 93–100% of the time, Model L only 71–81%.

And the thing I would take into the next project is that two Phase 1 decisions — a missing byte fallback and an exact-match-only deduplicator — both stayed invisible for two phases and then caused concrete problems. Corpus and tokenizer construction are experimental choices, not setup.

Appendix A

Final Cheat Sheet

The project in one sentence

Two independent ≈23M-parameter decoder-only Transformers built from scratch for Hindi and Nepali, with script and architecture held constant so that the resource-tier comparison isolates data and tokenizer effects.

A.1 Must-memorise numbers

Quantity	Model H	Model L
Corpus tokens	505,852,028	635,204,333
Documents	704,101	1,918,265
Manual share	30.66%	26.41%
Tokenizer / vocabulary	Unigram 8,000	BPE 10,000
Fertility	1.3956	1.4825
Unknown rate	0.000000	0.003387
Train tokens	485,280,741	519,906,359
Total parameters	23,273,472	24,297,472
Non-embedding parameters	18,915,328	18,915,328
Tokens trained	475,791,360	473,169,920
Evaluated checkpoint	step 14,500	step 14,000
Test perplexity	27.41	46.68
Test bits-per-byte	0.5382	0.4846
Bytes per token	8.875	11.441
Attention entropy	3.2047	3.0898
Attention distance	62.36	54.51
Exact match after finetuning	35.15%	29.45%
Forced choice after finetuning	36.35%	39.30%

Constants: uniform entropy baseline **5.2565**; chance rate **41.67%**; context **512**; effective batch **32,768 tokens**.

A.2 Formula sheet

Formula	Symbols	Why / example
$\text{softmax}(QK^\top/\sqrt{d_k}+M)V$	$d_k = 64$, M additive mask	Weighted blend of values by query–key relevance
$H = -\sum p \log p$	p = attention weights	How diffuse a head is; uniform over 2 gives 0.69 nats
$L = -\frac{1}{N}\sum \log p(x_t \mid x_{<t})$	N tokens	The training objective
$\text{PPL} = e^L$	L in nats	$e^{3.3109} = 27.41$
$b = L/\ln 2$	$\ln 2 = 0.6931$	$3.3109/0.6931 = 4.7766$
$\text{BPB} = b/(\text{bytes/token})$	—	$4.7766/8.875 = 0.5382$
$J(A,B) = \lvert A\cap B\rvert/\lvert A\cup B\rvert$	shingle sets	Near-duplicate similarity; $3/7 = 0.43$
$4(d^2{+}d) + 2d\,d_{ff} + d_{ff} + d + 4d$	d=512, d_{ff}=2048	Parameters per block = 3,152,384

A.3 Ten things I must NOT claim

1. That Hindi used MinHash. It did not — SHA-1 exact only.
2. That the models perform reasoning. Forced choice is at or below chance.
3. That Model L is a better language model because its BPB is lower. BPB is not corpus-neutral.
4. That attention analysis proves any head does the comparison. It is correlational.
5. That the ablation proves causal masking leaks position. Consistent with, not proven by.
6. That either pretraining run converged. Both stopped early with loss still falling.
7. That exact match improving means capability improved. Loss fell while forced choice fell too.
8. Any wall-clock comparison between the baseline and the ablation — different GPUs.
9. That the corpora are free of train/test leakage. Hindi's is unverified.
10. That the manual-share percentages are like-for-like. Different weighting per language.

Final conclusion

Hold the script and the architecture constant, and what is left is data and tokenizer — and that is where the differences turned out to live. The perplexity gap is mostly tokenizer granularity. The reasoning gain is mostly answer formatting. The resource tier shows up as groundedness, not reasoning. And the two Phase 1 decisions that caused real problems two phases later are the argument for treating corpus and tokenizer construction as experiments rather than setup.